

Design and Verification of a Parameterized UART Transmitter Core

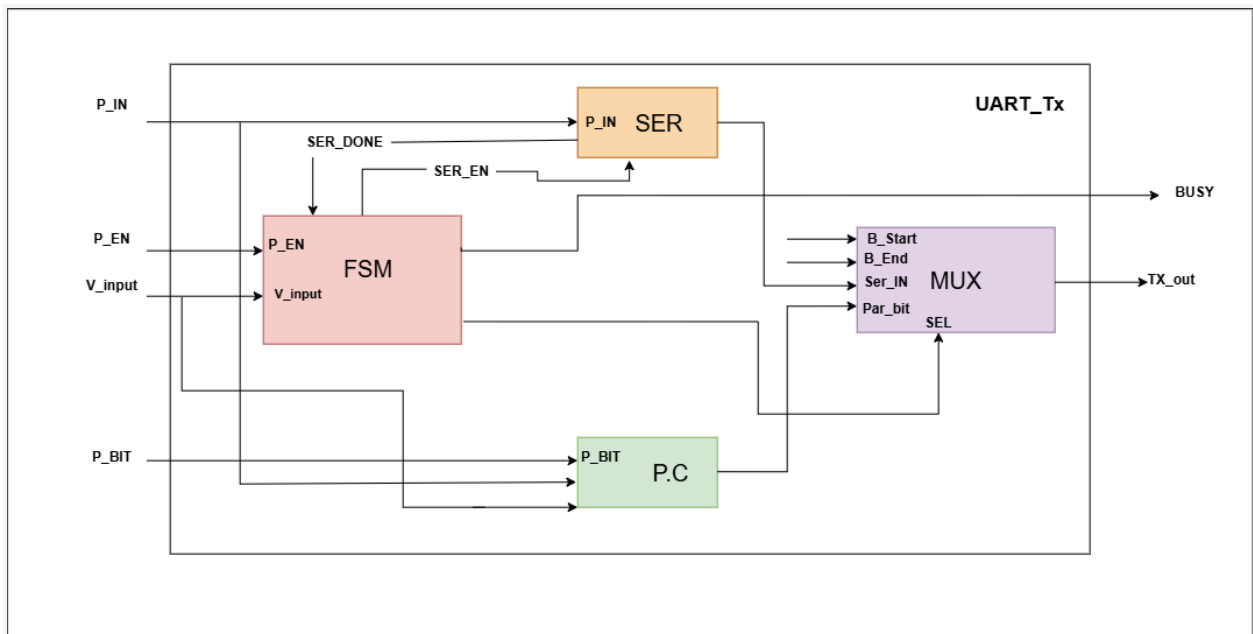
Abstract

This report details the design, implementation, and unit verification of a fully parameterized Universal Asynchronous Receiver-Transmitter (UART) Transmitter engine implemented in Verilog HDL. The architecture consists of four distinct sub-modules: a Finite State Machine (FSM) Controller, a Parallel-to-Serial Converter (Serializer), a Parity Calculator (P.C), and a 4-to-1 Multiplexer (MUX). Each constituent module was verified independently using a dedicated testbench before full top-level integration.

1. System Architecture Overview

The top-level UART_TX architecture transforms parallel data (P_IN) into a serial bitstream (TX_out) framed by standard Start (0), Parity (Par_bit), and Stop (1) bits.

Figure 1: Top-Level Architectural Block Diagram



Module Descriptions

Module Name	Identifier	Description
FSM Controller	FSM_control	Coordinates framing transitions and asserts system control signals (Ser_En, Sel, Busy) based on inputs P_EN, V_input, and Ser_done.
Serializer	Serializer	Performs parallel-to-serial conversion of data byte P_IN when enabled by Ser_En, generating Ser_done upon completion.
Parity Calculator	Par_calc	Calculates even or odd parity over the data byte P_IN as selected by configuration bit P_Bit.
Multiplexer	MUX	Routes the appropriate line—Start Bit (0), Serial Bit (Ser_IN), Parity Bit (Par_bit), or Stop Bit (1)—to output port TX_out based on selector code Sel.

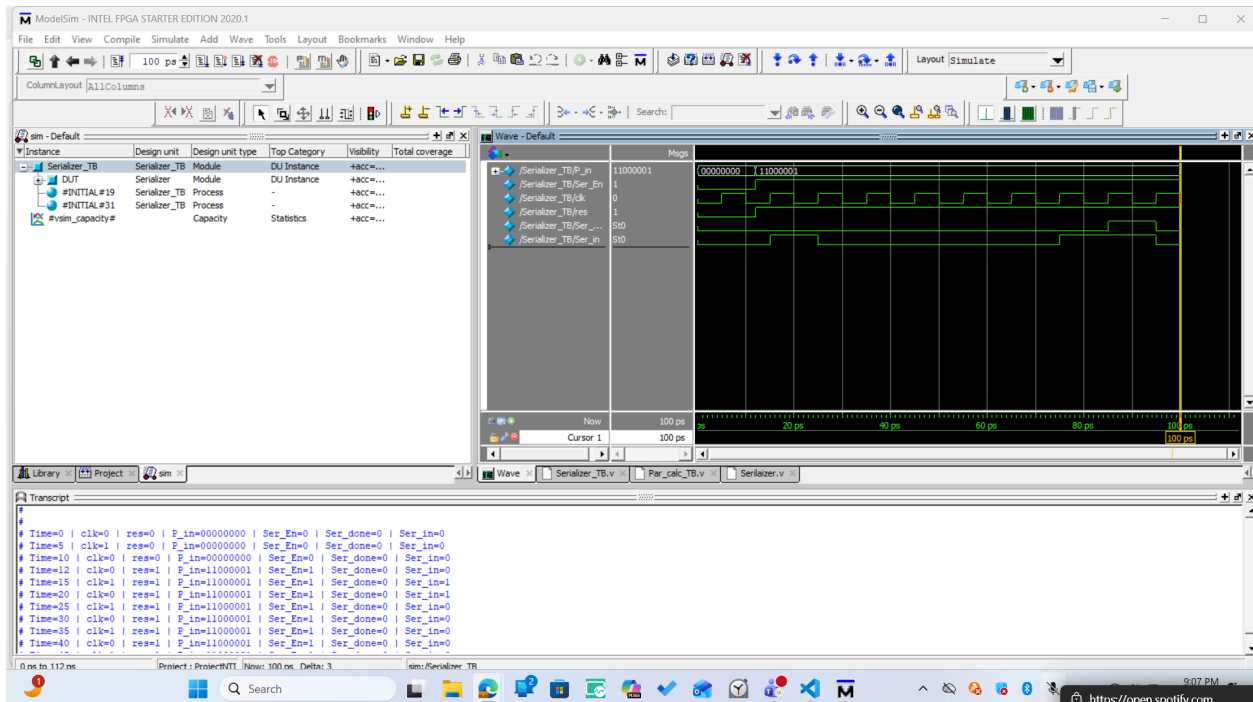
2. Unit Verification and Simulation Waveform Analysis

2.1 Serializer Sub-Module Verification

Figure 2: Serializer Module Waveform

The Serializer module converts parallel input data P_IN to a single-bit serial stream Ser_IN.

- **Execution Flow:** Upon asserting enable signal Ser_En = 1, the internal shift register loads P_IN = 8'b11000001. On subsequent clock cycles, bits are shifted out LSB-first over Ser_IN.
- **Waveform Commentary:** Ser_IN sequentially reflects bits 1 -> 0 -> 0 -> 0 -> 0 -> 0 -> 1 -> 1. Flag Ser_done pulses high upon completing the transfer sequence.



2.2 Multiplexer (MUX) Sub-Module Verification

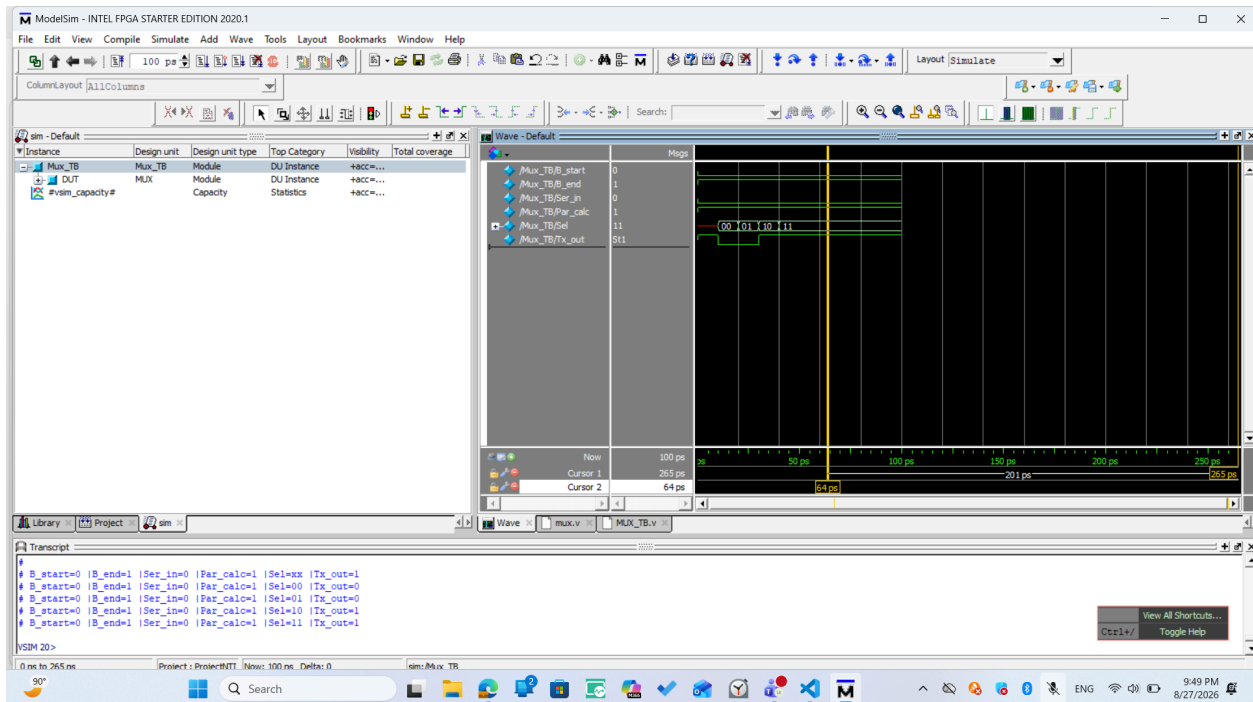
Figure 3: Multiplexer Waveform

The MUX module selects output line TX_out based on selector signal Sel[1:0].

Selection Mapping:

Selector Signal (Sel[1:0])	Routed Line	Output (Tx_out)
2'b00	Start Bit	0
2'b01	Stop Bit	1
2'b10	Calculated Parity Bit	Par_calc
2'b11	Serial Bit Stream	Ser_IN

Waveform Commentary: As Sel transitions from 00 through 11, output Tx_out mirrors the corresponding input line accurately without logic glitches.

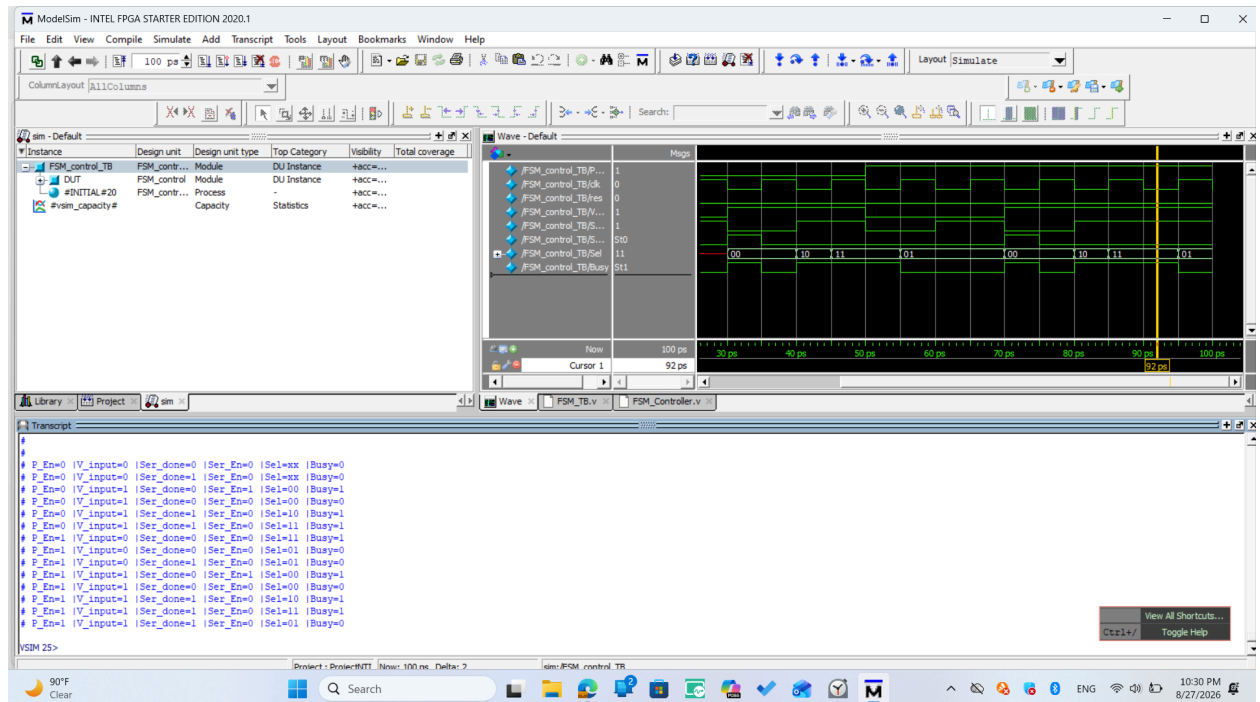


2.3 FSM Controller Sub-Module Verification

Figure 4: FSM Controller Waveform

The FSM controller governs state transitions across IDLE (00), DATA (11), PARITY (10), and STOP (01).

- **Execution Flow:** When valid transaction trigger $V_input = 1$ occurs in IDLE, state register transitions to DATA (Sel = 11) and asserts status line Busy = 1. Upon receiving Ser_done = 1, state advances to PARITY (Sel = 10), then STOP (Sel = 01), and back to IDLE (Busy = 0).
- **Waveform Commentary:** Signal Busy remains asserted throughout frame construction and de-asserts upon returning to IDLE.

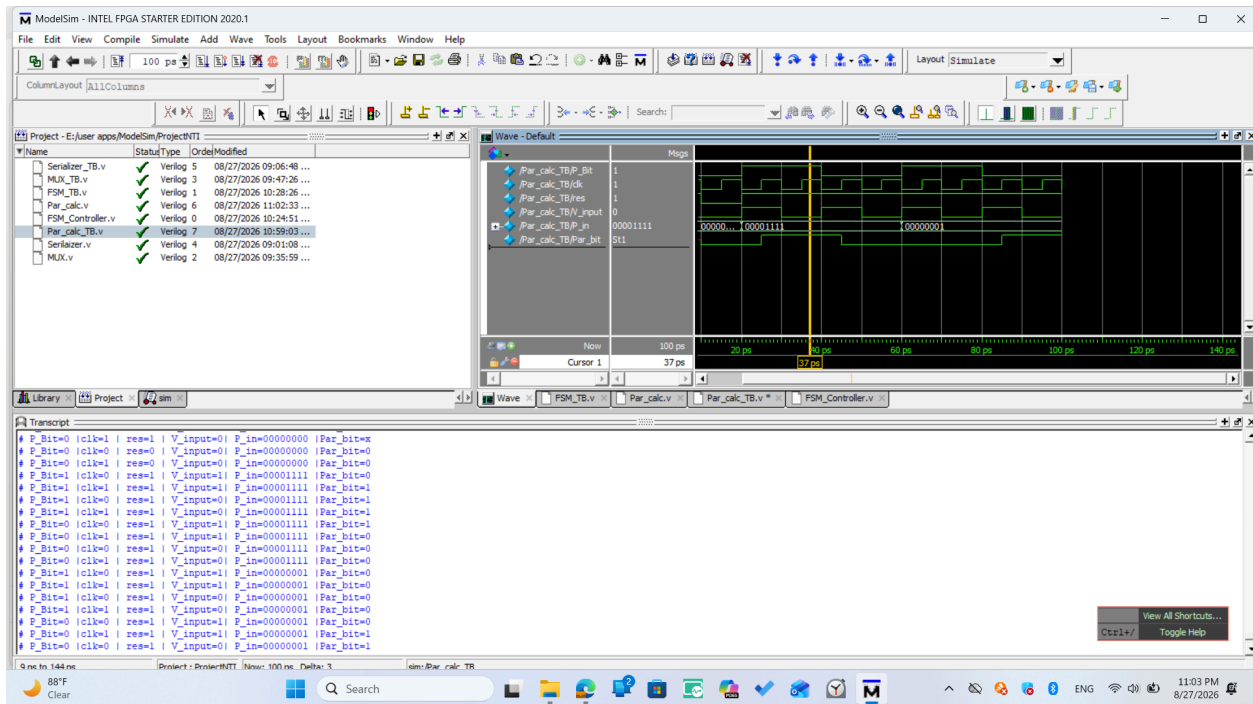


2.4 Parity Calculator Sub-Module Verification

Figure 5: Parity Calculator Waveform

The Parity Calculator derives parity status bit Par_bit based on mode flag P_Bit.

- **Execution Flow:** Mode P_Bit = 0 calculates Even Parity, while P_Bit = 1 calculates Odd Parity over P_IN.
- **Waveform Commentary:** For P_IN = 8'b00001111 (four set bits), Even Parity yields Par_bit = 0 and Odd Parity yields Par_bit = 1.



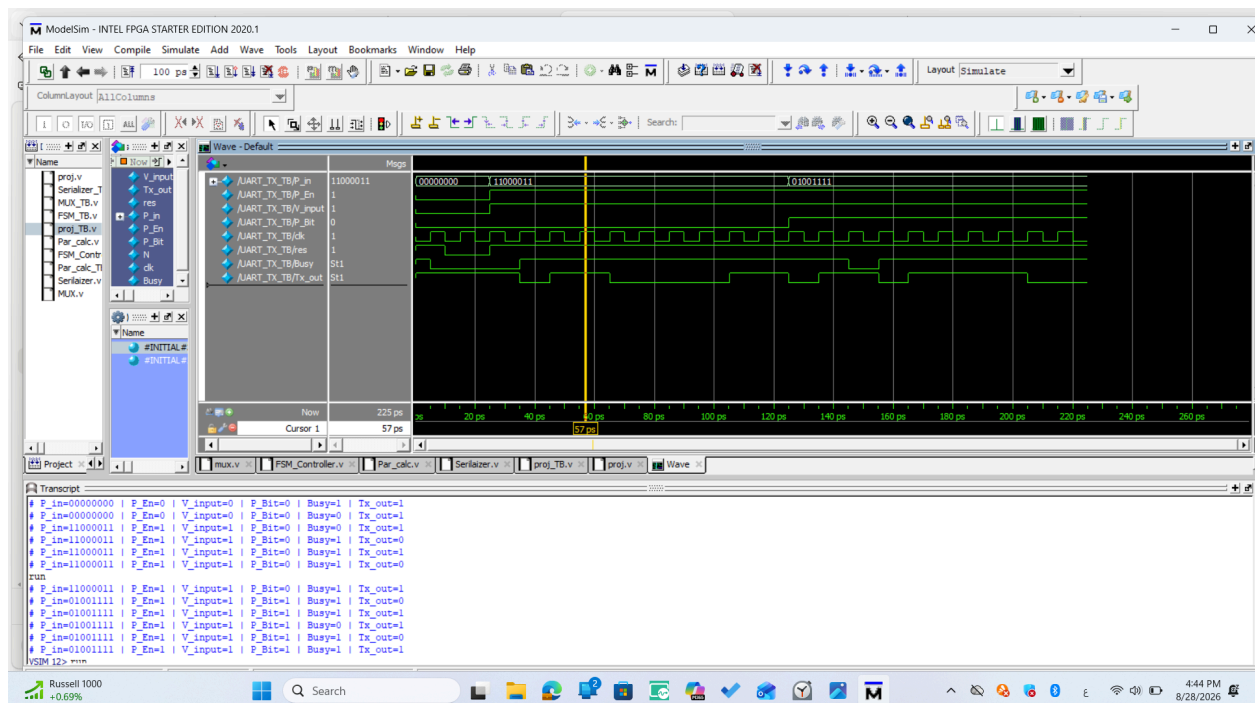
3. Top-Level Integration & Simulation Results

Figure 6: Integrated UART_TX Top Waveform

Full integration verification evaluates end-to-end frame serialization across complete transfer cycles.

Simulation Sequence:

1. **Idle State:** Inputs P_IN = 8'b11000011, P_Bit = 0. The system holds TX_out = 1 (STOP line) and Busy = 0.
2. **Transmission Start:** Asserting V_input = 1 triggers Busy = 1. The multiplexer outputs Start bit (0), followed by LSB-first serial data bits (1, 1, 0, 0, 0, 0, 1, 1).
3. **Parity & Framing Completion:** The frame completes with parity bit insertion (0 for Even Parity) and Stop bit (1), after which Busy returns to 0.



4. RX Unit Verification and Waveform Analysis

4.1 Negative Edge Detector Sub-Module Verification

Detects the transition from IDLE (1) to Start Bit (0).

4. **Execution Flow:** Uses double-stage flip-flops to sample RX_IN. Asserts a single-cycle pulse `neg_edge = 1` when a transition from high to low occurs. `neg_edge` triggers immediately following the high-to-low transition on RX_IN, initiating state traversal in FSM_RX.

4.2 Serial-to-Parallel (Deserializer) Verification

Converts incoming bit stream into an N -bit register word.

- **Execution Flow:** When enabled by the RX FSM, the internal counter counts bit durations and shifts RX_IN into an 8-bit internal buffer LSB-first. After 8 bit clock periods, parallel bus P_DATA updates to reflect the deserialized byte, and completion flag deser_done is asserted.

3.3 Parity Check Sub-Module Verification

Validates received parity against configured expectation.

- **Execution Flow:** Evaluates the reconstructed byte P_DATA together with the received Parity Bit bit position and if parity matches configured mode (Even/Odd), Parity_Err remains 0. If corrupted, Parity_Err flags 1 during validation stage.

3.4 Frame Check Sub-Module Verification

Ensures framing alignment by validating Stop Bit presence.

- **Execution Flow:** Inspects RX_IN during the designated Stop Bit sampling window and if RX_IN is high (1), Frame_Err remains 0. If line is sampled low (0), Frame_Err asserts 1 indicating a framing violation.

3.5 FSM_RX Sub-Module Verification

Manages state flow: IDLE -> START -> DATA -> PARITY -> STOP -> VALID.

- **Execution Flow:** Idle until the neg_edge pulse arrives. Sequentially activates Deserializer, Parity Check, and Frame Check modules. Asserts Data_Valid = 1 for one cycle if both error signals are 0 and Data_Valid pulses high only upon completion of framing checks without errors, returning the system to IDLE.

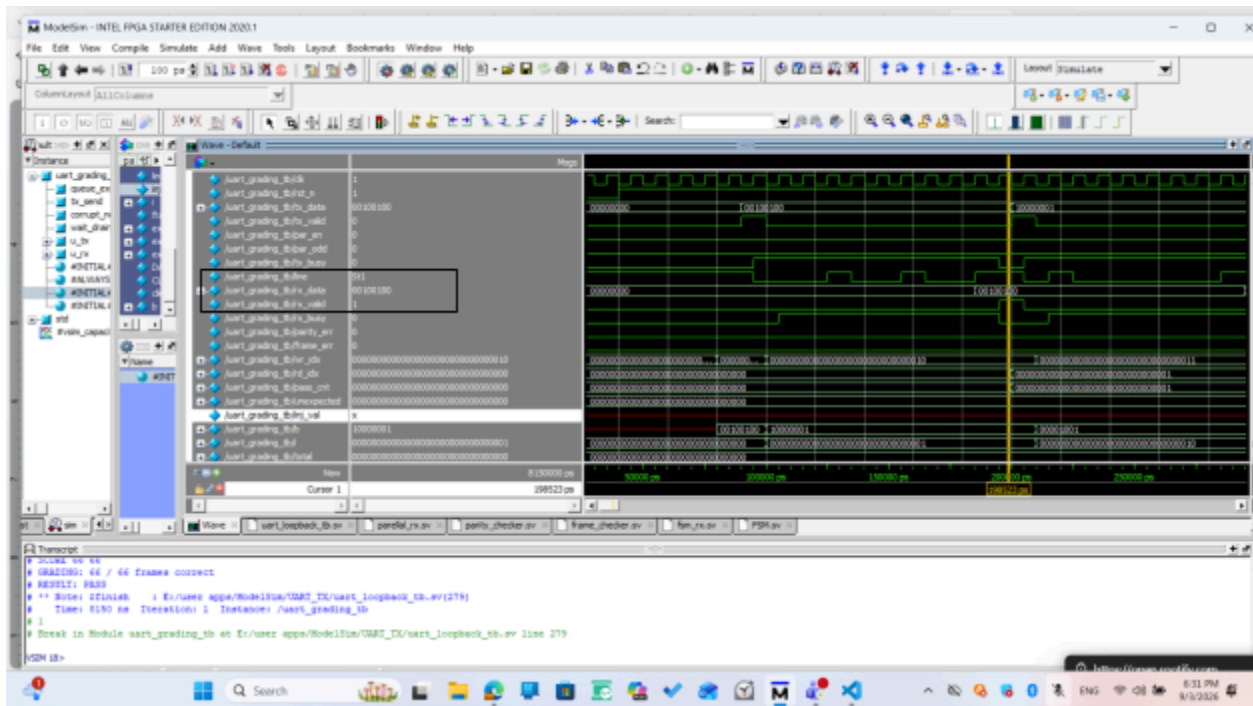
4. Top-Level Integration & Simulation Results

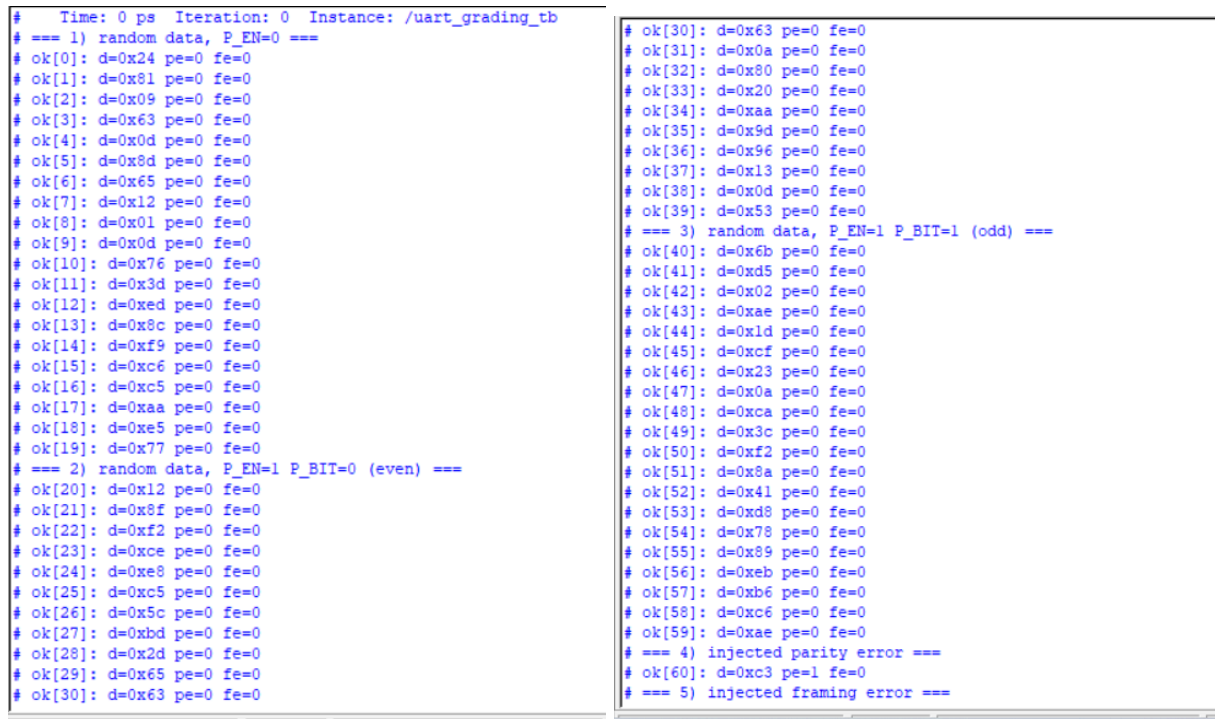
4.1 Integrated UART_TX Waveform

Full integration verification evaluates end-to-end frame serialization across complete transfer cycles.

Simulation Sequence:

1. **Idle State:** Inputs P_IN = 8'b11000011, P_Bit = 0. The system holds TX_out = 1 (STOP line) and Busy = 0.
2. **Transmission Start:** Asserting V_input = 1 triggers Busy = 1. The multiplexer outputs Start bit (0), followed by LSB-first serial data bits (1, 1, 0, 0, 0, 0, 1, 1).
3. **Parity & Framing Completion:** The frame completes with parity bit insertion (0 for Even Parity) and Stop bit (1), after which Busy returns to 0.





```

# ok[49]: d=0x3c pe=0 fe=0
# ok[50]: d=0xf2 pe=0 fe=0
# ok[51]: d=0x8a pe=0 fe=0
# ok[52]: d=0x41 pe=0 fe=0
# ok[53]: d=0xd8 pe=0 fe=0
# ok[54]: d=0x78 pe=0 fe=0
# ok[55]: d=0x89 pe=0 fe=0
# ok[56]: d=0xeb pe=0 fe=0
# ok[57]: d=0xb6 pe=0 fe=0
# ok[58]: d=0xc6 pe=0 fe=0
# ok[59]: d=0xae pe=0 fe=0
# === 4) injected parity error ===
# ok[60]: d=0xc3 pe=1 fe=0
# === 5) injected framing error ===
# ok[61]: d=0x5c pe=0 fe=1
# === 6) back-to-back frames ===
# ok[62]: d=0xa5 pe=0 fe=0
# ok[63]: d=0x5a pe=0 fe=0
# ok[64]: d=0xff pe=0 fe=0
# === 7) busy-reject ===
# ok[65]: d=0x39 pe=0 fe=0
#
# frames sent      : 66
# frames delivered : 66
# unexpected frames: 0
# SCORE 66 66
# GRADING: 66 / 66 frames correct
# RESULT: PASS
# ** Note: $finish      : E:/user apps/ModelSim/UART_TX/uart_loopback_tb.sv(279)
#      Time: 8150 ns  Iteration: 1  Instance: /uart_grading_tb
# 1
# Break in Module uart_grading_tb at E:/user apps/ModelSim/UART_TX/uart_loopback_tb.sv line 279
VCSIM 18>

```

5. Conclusion

The RTL implementation of both parameterized UART_TX and UART_RX cores was synthesized and verified successfully. Unit testing confirmed functionality across all sub-modules (Serializer, MUX, FSM_control, Par_calc, neg_edge_det, deserializer, parity_chk, frame_chk, and FSM_RX), and top-level integration verified compliance with standard serial transmission and reception protocols.